Hibernate Core Basic API

Table of Contents

1.Introduction.....1
2.Architecture 1
3.Core API.....2
4.Extension Interfaces.....6
5.Sample Hibernate Session.....6

1. Introduction

Hibernate is an Object-Relational Mapping(ORM) solution for JAVA and it raised as an open source persistent framework created by Gavin King in 2001. It is a powerful, high performance Object-Relational Persistence and Query service for any Java Application.

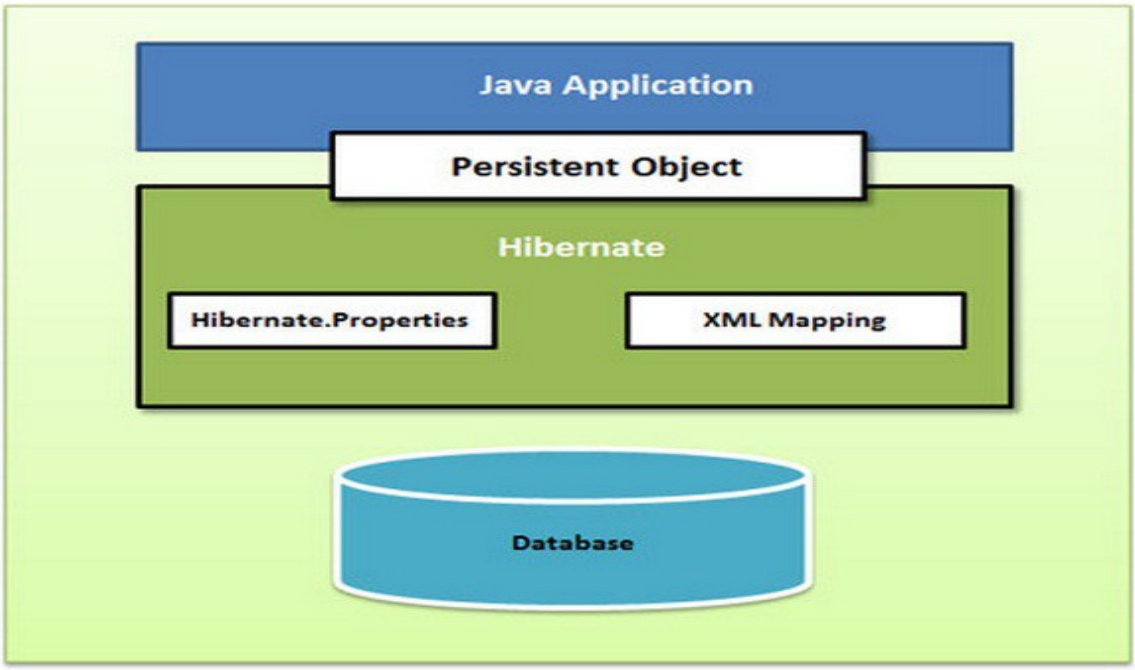
Hibernate maps Java classes to database tables and from Java data types to SQL data types and relieve the developer from 95% of common data persistence related programming tasks.

Hibernate sits between traditional Java objects and database server to handle all the work in persisting those objects based on the appropriate O/R mechanisms and patterns.

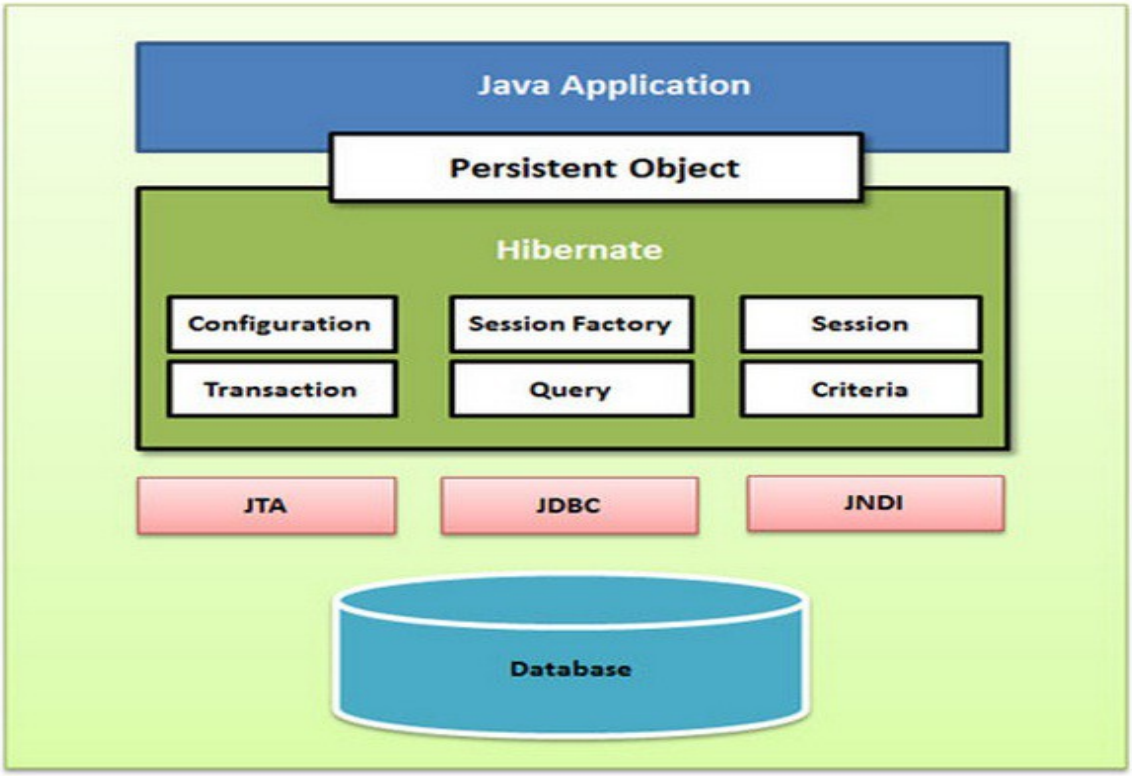
2. Architecture

- ⇧ Hibernate is a pure Java object-relational mapping (ORM) that allows to map plain old Java objects(POJO) to relational database tables using (XML) configuration files or by annotation methods
- ⇧ Provides a simple API for storing and retrieving Java objects directly from the database
- ⇧ This is a ORM framework that allows to store the object states in to RDBMS tables efficiently with minimal amount of boiler plate code.
- ⇧ Provides an abstraction layer between the domain model and the database

a) **High Level View of Hibernate Application Architecture**



b) **Detailed view of the Hibernate Application Architecture with few core classes.**



3. Core API

This documentation concentrates upon the following Core API interfaces:

- `org.hibernate.Hibernate`
- `org.hibernate.Session`
- `org.hibernate.SessionFactory`
- `org.hibernate.Transaction`
- `org.hibernate.Query`
- `org.hibernate.Criteria`
- `org.hibernate.ScrollableResults`
- `org.hibernate.cfg.Configuration`
- `org.hibernate.expression.Expression`
- `org.hibernate.expression.Order`
- org.hibernate.expression.Example

a) Session (org.hibernate.Session)

- The Session interface is the primary interface used by Hibernate applications.

- An instance of Session is lightweight and is inexpensive to create and destroy.This is important because your application will need to create and destroy sessions all the time, perhaps on every request.

- Hibernate sessions are not thread-safe and should by design be used by only one thread at a time.

- The Hibernate notion of a session is something between connection and transaction.

  Moreover, its a single-threaded, short-lived object representing a conversation between the application and the persistent store.Wraps a JDBC java.sql.Connection.Factory for org.hibernate.Transaction. Maintains a first level cache of persistent between persistent objects and collections; which is used when navigating the object graph or looking up objects by identifier.

  SessionImpl is implemented class of Session Interface. When ever session is opened then internally a database connection will be opened, in order to get a session or open a session we need to call openSession() method in SessionFactory, it means SessionFactory produces sessions.

Session	openSession() Create database connection and open a Session on it.
Session	openSession(Connection connection) Open a Session on the given connection.
Session	openSession(Connection connection, Interceptor interceptor) Open a Session on the given connection, specifying an interceptor.
Session	openSession(Interceptor interceptor) Create database connection and open a Session on it, specifying an interceptor.

```
Session session = sessionFactory.openSession();
```

Method	Operation
Session.save(emp)	Insert object 'emp' in the database
Session.update(emp)	Update object 'emp' in the database
Session.load(emp)	Selecting object 'emp' from database
Session.delete(emp)	Deleting object 'emp' from database
Session.close()	Ends the session by releasing datbase connection

b) **SessionFactory (org.hibernate.SessionFactory)**

A thread-safe, immutable cache of compiled mappings for a single database. A factory for org.hibernate.Session instances. A client of org.hibernate.connection.ConnectionProvider. Optionally maintains a second level cache of data that is reusable between transactions at a process or cluster level.

- ⇧ The application obtains Session instances from a Session-factory. Compared to the Session interface, this object is much less exciting.

- ⇧ The Session-factory is certainly not lightweight! It's intended to be shared among many application threads. There is typically a single Session-factory for the whole application,created during application initialization.

- ⇧ The Session Factory caches generated SQL statements and other mapping metadata that Hibernate uses at run-time. It also holds cached data that has been read in one unit of work and may be reused in a future unit of work (only if class and collection mappings specify that this second-level cache is desirable).

- ⇧ SessionFactoryImpl is the implimented class for SessionFactory Interface so we are internally creating object of SessionFactoryImpl class and storing in the interface reference, so this SessionFactory object sessionFactory contains all the data regarding the configuation file.

c) **Configuration interface**

The Configuration object is used to configure and bootstrap Hibernate. The application uses a Configuration instance to specify the location of mapping documents and Hibernate-specific properties and then create the Session Factory.

Among Configuration, Mapping xml files, first we need to load configuration xml, because once we load the configuration file, automatically mapping file will be loaded as we registered this mapping xml in the configuration file.
So to load configuration xml, we need to create object of Configuration class, and we need to call configure() method in that class, by passing xml configuration file name as parameter. (if no parameter is provided then default configuration file is hibernate.cfg.xml)

Configuration	configure() Use the mappings and properties specified in an application resource named hibernate.cfg.xml.
Configuration	configure(org.w3c.dom.Document document) Use the mappings and properties specified in the given XML document.
Configuration	configure(File configFile) Use the mappings and properties specified in the given application file.
Configuration	configure(String resource) Use the mappings and properties specified in the given application resource.
Configuration	configure(URL url) Use the mappings and properties specified in the given document.

d) **Transaction (org.hibernate.Transaction)**

- The Transaction interface is an optional API. Hibernate applications may choose not to use this interface, instead managing transactions in their own infrastructure code.

- A Transaction abstracts application code from the underlying transaction implementation—which might be a JDBC transaction

Moreover, its a single-threaded, short-lived object used by the application to specify atomic units of work. It abstracts the application from the underlying JDBC, JTA or CORBA transaction. A org.hibernate.Session might span several org.hibernate.Transactions in some cases. However, transaction demarcation, either using the underlying API or org.hibernate.Transaction, is never optional.

e) **Query and Criteria interfaces**

- The Query interface allows you to perform queries against the database and control how the query is executed. Queries are written in HQL or in the native SQL dialect of your database.

- A Query instance is used to bind query parameters, limit the number of results returned by the query, and finally to execute the query.

- The Criteria interface is very similar; it allows you to create and execute object-oriented criteria queries.

f) **ConnectionProvider (org.hibernate.connection.ConnectionProvider)**

A factory for, and pool of, JDBC connections.It abstracts the application from underlying javax.sql.DataSource or java.sql.DriverManager.It is not exposed to application, but it can be extended and/or implemented by the developer.

g) **TransactionFactory (org.hibernate.TransactionFactory)**

(Optional) A factory for org.hibernate.Transaction instances. It is not exposed to the application, but it can be extended and/or implemented by the developer.

4. Extension Interfaces

The Extension API is intended to be used by application programmers to extend Hibernate functionality. None of these interfaces are intended to be called by the application - they are called internally by Hibernate. This API is less stable than the Core API. The safest way to extend functionality is to contribute extensions back to the project; that way extensions will be updated when the Extension API changes.

Hibernate offers a range of optional extension interfaces you can implement to customize the behavior of your persistence layer. See the API documentation for details.

For further information :

http://docs.jboss.org/hibernate/orm/3.2/api/

5. Sample Hibernate Session

Java - Hibernate_Demo/src/pd/practice/hibernate/Helper/SessionHelper.java - Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

TestManytoMany.java | SessionHelper.java

```java
package pd.practice.hibernate.Helper;

import org.hibernate.Session;

public class SessionHelper {
    public Session getHibernateSession() {
        Configuration configuration = new Configuration().configure();
        ServiceRegistryBuilder registryBuilder = new ServiceRegistryBuilder()
                .applySettings(configuration.getProperties());
        ServiceRegistry serviceRegistry = registryBuilder
                .buildServiceRegistry();
        SessionFactory sessionFactory = configuration
                .buildSessionFactory(serviceRegistry);
        Session session = sessionFactory.openSession();
        System.out.println("the session object :: " + session);
        return session;
    }

    public void closeHibernateSession(Session session) {
        session.close();
        System.out.println("Session successfully closed");
    }

}
```